Project 3: Consumer Lending — Student Guide

The question: Why are approvals falling and early defaults rising?

You lead this one too. Use the same four phases as Projects 1 and 2, and reuse your earlier files as templates.

Some things to know before you start:

- There are **two headlines**: approvals falling, and early defaults rising. Don't assume they have the same cause.
- There is **more than one cause**.
- There is at least **one tempting explanation that is wrong**.
- No single query gives you the answer.

Contents

1. What you will hand in
2. Before you start
3. Words you will see
4. Meet the data
5. Definitions to use
6. Phase 1: SQL
7. Phase 2: Excel
8. Phase 3: Python
9. Phase 4: Power BI
10. Phase 5: The verdict
11. Checkpoints and grading
12. Common mistakes
13. SQL Server 2016 cheat sheet
14. When you get stuck

---

1. What you will hand in

#	Deliverable	Made in
1	A `.sql` file with commented queries and row-count checks	SQL portal or SSMS
2	An Excel workbook: audit sheet, issue log, pivots, hypotheses	Excel
3	A notebook (`.ipynb`): cleaning, checks, hypothesis tests, CSV export	Colab or Jupyter
4	Star-schema CSV files	Python
5	A Power BI report, 3 pages plus an About page: a `.pbix` file, or a report link plus PDF if you use the browser	Power BI
6	A one-page verdict	Word or PDF

The one-page verdict matters most. A credit manager should be able to read it and know what to change.

---

2. Before you start

Choose your tools

Use the same tools you chose for Project 1. The Project 1 guide (section 2) has the full setup steps for each one.

Phase	Option A	Option B
SQL	**Online SQL portal** at `ssms.penguins.digital`	**SSMS** on Windows
Excel	**Excel for the web** or Microsoft 365	**An older Excel**, such as 2016 or 2019
Python	**Google Colab** in the browser	**Jupyter** on your own computer (pandas 2.0 or later)
Power BI	**Power BI Desktop** on Windows	**Power BI in the browser** on a Mac

Quick reminders:

- **Portal:** it runs everything in the query tab and shows only the first result, so keep one query per tab. The grid shows up to 500 rows; **Export CSV** saves every row. Queries time out after 60 seconds.
- **Older Excel:** no `UNIQUE` , `XLOOKUP` or `FILTER` . Use **Remove Duplicates** or a PivotTable for distinct values, and `INDEX` / `MATCH` for lookups.
- **Colab:** put `capstone_lending/` in Google Drive and start your notebook with:

```
from google.colab import drive
drive.mount("/content/drive")
%cd /content/drive/MyDrive/capstone_lending
```

- **Jupyter:** `pip install pandas numpy openpyxl jupyter`
- **Power BI on a Mac:** use a Windows lab PC, or Power BI in the browser at `app.powerbi.com` with your school account. In the browser, upload your star schema as one Excel workbook, and hand in a report link plus a PDF instead of a `.pbix` file.

Set up your folder

```
capstone_lending/
    lending_queries.sql
    lending_audit.xlsx
    lending_analysis.ipynb
    csv/
    lending_report.pbix
    lending_verdict.docx
```

Test your connection

```
SELECT COUNT(*) AS application_count FROM dbo.lending_applications;
```

You should see **4,000**.

---

3. Words you will see

Lending has its own vocabulary. You don't need a finance background. These are the words that matter:

Word	Plain meaning
Application	A request for a loan. It can be approved, rejected, withdrawn by the applicant, or still pending.
Loan	Money actually paid out. Only approved applications become loans.
Disbursement	The day the money is paid out to the borrower.
Installment	One monthly payment. A 36-month loan has 36 installments.
Days past due (DPD)	How many days late a payment is. 30+ DPD means at least 30 days late.
Default	The borrower stops paying.
Early default	A loan that goes bad soon after it starts (see the definitions). It's a sign the loan shouldn't have been made.
Credit score	A number from 300 to 850. Higher means the person has a better track record of paying back.
DTI (debt-to-income)	Monthly debt payments divided by monthly income. A DTI of 0.40 means 40% of income already goes to debt. Higher is riskier.
Credit policy	The lender's rules, such as a minimum credit score or a maximum DTI for each product.
Policy exception	A loan approved outside the rules.
Tenor	How long the loan lasts, in months.
Vintage	All loans paid out in the same period, such as "the 2024 Q1 vintage."
Seasoned loan	A loan that is old enough to judge. A loan paid out last week can't have missed six payments yet.
Collections	The team or agency that chases late payments.
Channel	How the application arrived: in a **Branch**, through an **Agent**, or **Online**.
Grain	What one row means. In `lending_repayments`, one row is one installment of one loan, so one loan has many rows.

---

4. Meet the data

A US consumer lender with 25 branches and 4 loan products, from January 2023 to December 2025.

Table	Rows	One row is	Type
`lending_branches`	25	A branch	Core
`lending_officers`	120	A loan officer	Core
`lending_products`	4	A loan product and its policy limits	Core
`lending_borrowers`	3,050	A borrower and their financial profile	Core
`lending_applications`	4,000	A loan application and its decision	Core
`lending_loans`	2,008	A loan that was paid out	Core
`lending_repayments`	33,166	**One installment of one loan**	Core
`lending_collections_feed`	268	A case from the outside collections agency	**Feed**

The four products are **Personal**, **Auto**, **SME** (small business) and **Home**. Each has its own limits in `lending_products`: a lowest allowed credit score (`min_credit_score`), a highest allowed DTI (`max_dti`), a rate range, and a longest tenor.

How the tables connect

```
lending_branches ──< lending_officers
lending_branches ──< lending_applications >── lending_officers
lending_borrowers ──< lending_applications >── lending_products
lending_applications ─── lending_loans        (at most one loan per application)
lending_loans ──< lending_repayments          (one row per installment)

lending_collections_feed ·····> lending_loans.loan_number   (text match, not a key)
```

To get from	To	Join on
`lending_applications`	`lending_borrowers`	`borrower_id`
`lending_applications`	`lending_products`	`product_id`
`lending_applications`	`lending_branches`	`branch_id`
`lending_applications`	`lending_officers`	`officer_id`
`lending_loans`	`lending_applications`	`application_id`
`lending_repayments`	`lending_loans`	`loan_id`
`lending_officers`	`lending_branches`	`branch_id`
`lending_collections_feed`	`lending_loans`	`loan_ref_text` = `loan_number`, after cleaning

The full list of columns is in the lending data dictionary (`capstone/docs/lending_data_dictionary.md`).

Things that trip people up

- **Repayments have many rows per loan.** Joining them straight to applications multiplies your rows. **Add up repayments to one row per loan in SQL first**, then join.
- **Some loans have no repayment rows yet.** Loans paid out at the very end of 2025 have no installments due. An `INNER JOIN` to repayments drops them without warning.
- To get from a loan to its branch, product or borrower, go **through** `lending_applications`.
- Rates, DTI and percentages are stored as **fractions**: `0.1250` means 12.5%. DTI can be over 1.
- `decline_reason` is only filled in for rejected applications, and not always then.

---

5. Definitions to use

Everyone in class uses these. Write them at the top of your `.sql` file.

Metric	Definition
Approval rate	Approved / (Approved + Rejected). Leave out Withdrawn and Pending.
Early default	Any of the first 6 installments (`installment_no` 1 to 6) reaches 30 or more `days_past_due`
Seasoned loan	`DATEADD(DAY, 30, DATEADD(MONTH, 6, disbursement_date)) <= '2025-12-31'`

- **Only count seasoned loans** when you measure early default. Newer loans haven't had time to go bad, so including them makes recent results look better than they are.
- For approval rate, group by **application date**. For early default, group by **disbursement date**, unless you have a reason to do otherwise. Say which one you used.
- Show the group size next to every rate.

---

6. Phase 1: SQL: build a trusted extract

Step 1: Count the rows

```
SELECT 'applications' AS table_name, COUNT(*) AS row_count FROM dbo.lending_applications
UNION ALL SELECT 'loans', COUNT(*) FROM dbo.lending_loans
UNION ALL SELECT 'repayments', COUNT(*) FROM dbo.lending_repayments
UNION ALL SELECT 'borrowers', COUNT(*) FROM dbo.lending_borrowers
UNION ALL SELECT 'collections_feed', COUNT(*) FROM dbo.lending_collections_feed;
```

✅ **Check:** 4,000 applications, 2,008 loans, 33,166 repayments, 3,050 borrowers and 268 feed rows.

Step 2: Look at the decisions

```
SELECT decision, COUNT(*) AS applications
FROM dbo.lending_applications
GROUP BY decision
ORDER BY applications DESC;
```

✅ **Check:** Approved 2,089, Rejected 1,691, Withdrawn 193 and Pending 27.

Step 3: Headline 1: approval rate by year

```
SELECT YEAR(application_date) AS application_year,
       SUM(CASE WHEN decision = 'Approved' THEN 1 ELSE 0 END) AS approved,
       SUM(CASE WHEN decision = 'Rejected' THEN 1 ELSE 0 END) AS rejected,
       CAST(1.0 * SUM(CASE WHEN decision = 'Approved' THEN 1 ELSE 0 END)
            / SUM(CASE WHEN decision IN ('Approved', 'Rejected') THEN 1 ELSE 0 END)
            AS DECIMAL(6,4)) AS approval_rate
FROM dbo.lending_applications
GROUP BY YEAR(application_date)
ORDER BY application_year;
```

✅ **Check:** about **0.6413** in 2023, **0.5190** in 2024 and **0.5180** in 2025.

Notice: the `1.0 *` matters. Without it, SQL Server does whole-number division and every rate comes out as 0.

Step 4: See the repayments grain

```
SELECT COUNT(*) AS repayment_rows,
       COUNT(DISTINCT loan_id) AS loans_with_repayments
FROM dbo.lending_repayments;
```

✅ **Check:** 33,166 rows but only 1,948 loans. That's about 17 rows per loan. So 60 of the 2,008 loans have no repayment rows yet.

Try this: find those 60 loans with a `LEFT JOIN` and `WHERE r.loan_id IS NULL`. When were they paid out?

Step 5: Headline 2: early default by year

This query is given to you, because the definition has several parts. Read it one step at a time.

```
-- Step A: one row per seasoned loan, with a 1/0 early-default flag
WITH loan_status AS (
    SELECT l.loan_id,
           l.disbursement_date,
           MAX(CASE WHEN r.installment_no <= 6 AND r.days_past_due >= 30
                    THEN 1 ELSE 0 END) AS early_default
    FROM dbo.lending_loans AS l
    JOIN dbo.lending_repayments AS r ON r.loan_id = l.loan_id
    WHERE DATEADD(DAY, 30, DATEADD(MONTH, 6, l.disbursement_date)) <= '2025-12-31'
    GROUP BY l.loan_id, l.disbursement_date
)
-- Step B: the rate, by year the loan was paid out
SELECT YEAR(disbursement_date) AS disbursed_year,
       COUNT(*) AS seasoned_loans,
       SUM(early_default) AS early_defaults,
       CAST(1.0 * SUM(early_default) / COUNT(*) AS DECIMAL(6,4)) AS early_default_rate
FROM loan_status
GROUP BY YEAR(disbursement_date)
ORDER BY disbursed_year;
```

✅ **Check:** about **0.0285** in 2023, **0.0590** in 2024 and **0.0776** in 2025.

Why `MAX`? Each loan has many installment rows. `MAX` of the 1/0 flag is 1 if **any** of the first six installments was 30+ days late. That turns many rows into one row per loan.

Why only 322 loans in 2025? Only loans paid out early in 2025 are seasoned by the end of the year.

Your turn: Step A gives you one row per loan. Join it to `lending_applications` with `application_id` (add it to Step A), and you can break early default down by anything on the application.

Step 6: Look at the collections feed before joining it

```
SELECT TOP 50 *
FROM dbo.lending_collections_feed
ORDER BY NEWID();
```

```
SELECT dpd_bucket_text, COUNT(*) AS rows
FROM dbo.lending_collections_feed
GROUP BY dpd_bucket_text
ORDER BY rows DESC;
```

Look for:

- The same bucket written many ways: `30–59`, `30 to 59 days`, `30–59 DPD`, `90+`, `90 plus`
- Amounts with `USD` and without

- Loan references with and without a dash, or with extra spaces
- Dates in several formats
- The same `case_ref` more than once, and blank rows

Step 7: Save your extracts

Extract	Grain	Ideas for columns
Applications	One row per application	Application, date, channel, decision, decline reason, amount, tenor, DTI, exception flag, borrower profile, product and its limits, branch, region, officer
Loans	One row per loan	Loan, application, disbursement date, principal, rate, tenor, status, **seasoned flag**, **early-default flag**
Borrowers	One row per borrower	Every column
Collections feed	One row per feed row, untouched	Every column, as is

Do the adding up in SQL. Your loans extract should come from a query like Step 5, not from all 33,166 repayment rows.

Saving extracts: in the portal, click **Export CSV** (it saves every row). In SSMS, right-click the grid and choose **Save Results As...**. Save into your `csv/` folder, in Google Drive if you use Colab.

✅ **Check:** applications extract has 4,000 rows and loans extract has 2,008 rows. If you have fewer loans, you used an `INNER JOIN` where you needed a `LEFT JOIN`.

---

7. Phase 2: Excel: audit and explore

Audit first

Careful with the collections feed in Excel. Excel can reformat the mixed dates when it opens a CSV, which hides the problem you're auditing. In desktop Excel, use **Data > From Text/CSV**, choose **Transform Data**, set the feed columns to **Text**, then load. In Excel for the web, check a few cells against the SQL grid; if values were changed, audit that column in Python instead.

Places to look:

- **Borrowers:** the same person entered twice with small differences; city names spelled several ways; incomes that look like typos (far too high)
- **Applications:** rejected applications with no `decline_reason`
- **Collections feed:** duplicates, blanks, orphans, mixed formats, `N/A` amounts

Issue log

#	Table	Column	Problem	How many rows	What I will do
1	collections_feed	dpd_bucket_text	Same bucket written 4+ ways	?	Map to 30-59, 60-89, 90+
2	...	...	...	...	...

Every issue needs a **count** and a **decision**.

Explore with pivots

1. **Both headlines:** approval rate by application year, early default rate by disbursement year.
2. **Who is applying?** Count of applications by channel and year. Average credit score and DTI by channel.
3. **Where?** Approval rate and early default rate by branch and region.
4. **What?** The same by product.
5. **When?** By quarter. When did each change start?

Write your hypotheses

Write **at least 4** for each headline. Angles to consider:

- **Who applies:** credit score, DTI, income, employment type, age
- **How they apply:** channel
- **Where:** branch, region, city
- **Who decides:** loan officer
- **What they borrow:** product, amount, tenor
- **The rules:** were approvals inside each product's credit policy?
- **Time:** when did it start, and is it steady or getting worse?

Include the obvious suspects. Part of your grade is showing which explanations are wrong.

---

8. Phase 3: Python: clean and test

Load and clean

```
import pandas as pd

apps = pd.read_csv("csv/applications.csv", parse_dates=["application_date"])
loans = pd.read_csv("csv/loans.csv", parse_dates=["disbursement_date"])
feed = pd.read_csv("csv/collections_feed.csv", dtype=str)   # keep the feed as text for now

print(apps.shape, loans.shape, feed.shape)
```

Cleaning the collections feed:

1. Drop rows where everything is blank.
2. Clean `case_ref` (strip spaces, upper case), then drop duplicates.
3. Clean `loan_ref_text` so it matches `loan_number`.
4. Map `dpd_bucket_text` to a small set of clean buckets.
5. Turn `recovered_amount_text` into a number (remove `USD`, `$` and commas).
6. Parse `last_contact_date_text` with `pd.to_datetime(..., format="mixed", errors="coerce")`.

For borrowers, decide how you'll spot the same person entered twice. Which columns should match?

Prove it worked:

```
assert feed["case_ref"].is_unique
assert loans["loan_id"].is_unique
assert len(apps) == 4000
```

Test each hypothesis

For every hypothesis, write:

1. The hypothesis
2. The code that tests it
3. The result, **with group sizes**
4. **Supported** or **ruled out**, and why

Tips for this project:

- **Test the two headlines separately.** Something can explain one and have nothing to do with the other.
- **Compare like with like.** If you compare two groups, ask whether they differ in other ways too.
- **Look at rates, not totals.** A group with many defaults may simply have many loans.
- **Watch small groups.** At this size, one or two defaults can move a small group's rate by several points. Look at several periods together, not one quarter at a time.

Size it

- For approvals: how many more approvals would there have been without each cause?
- For defaults: how many early defaults, and how much principal, came from each cause?

Export a star schema

File	Type	One row is
`fact_applications.csv`	Fact	An application
`fact_loans.csv`	Fact	A loan, with seasoned and early-default flags
`dim_borrower.csv`	Dimension	A borrower, cleaned
`dim_product.csv`	Dimension	A product, with its policy limits
`dim_branch.csv`	Dimension	A branch, with its region
`dim_officer.csv`	Dimension	An officer
`dim_date.csv`	Dimension	A calendar day

---

9. Phase 4: Power BI: model and report

1. **Load** your star-schema CSVs and connect each fact to each dimension, one to many. In the browser on a Mac, upload them as one Excel workbook instead, with one sheet per table (`df.to_excel` with `pd.ExcelWriter` , as in the Project 1 guide).
2. **Measures:**

```
Approved = CALCULATE(COUNTROWS(fact_applications), fact_applications[decision] =
"Approved")
Rejected = CALCULATE(COUNTROWS(fact_applications), fact_applications[decision] =
"Rejected")
Approval Rate = DIVIDE([Approved], [Approved] + [Rejected])

Seasoned Loans = CALCULATE(COUNTROWS(fact_loans), fact_loans[is_seasoned] = 1)
Early Default Rate = DIVIDE(CALCULATE(SUM(fact_loans[early_default]),
fact_loans[is_seasoned] = 1), [Seasoned Loans])
```

3. **Reconcile:** check that both headline rates by year match your SQL and Python numbers **before** you build charts.
4. **Build 3 pages plus About:**
    - Page 1: the executive summary. Both headlines and their causes.
    - Page 2: why approvals fell.
    - Page 3: why early defaults rose.
    - About: sources, definitions, seasoning rule, cleaning steps and limits.

Report checklist

- [ ] Chart titles state the finding, not just the measure

- [ ] One highlight color for the problem, gray for everything else
- [ ] Every rate shows its group size
- [ ] The About page explains why only seasoned loans are counted

---

10. Phase 5: Write the verdict

One page. Answer first. Answer **both** headlines.

Section	What to write
Answer	Why approvals fell, and why early defaults rose. One or two sentences each.
Evidence	The two or three numbers that prove each one.
Size	Approvals lost, early defaults and dollars at risk from each cause.
Ruled out	The tempting explanations that turned out to be wrong, and why.
Action	Specific next steps, with an owner.
Risks and limits	What you couldn't check, and what could change your answer.

---

11. Checkpoints and grading

Checkpoints

#	Show your instructor
1	Your metric definitions and extract row counts
2	Your issue log and at least 4 written hypotheses for each headline
3	Clean numbers next to raw numbers, with the differences explained
4	A Power BI table that matches your Python numbers
5	A draft verdict, for peer review

Grading (100 points)

Area	Points	Excellent looks like
SQL	20	Correct grain, repayments aggregated first, right joins, row checks
Excel audit	15	Every issue counted, with a decision for each
Python	20	Repeatable, with asserts; hypotheses tested and some ruled out
Power BI	20	A sound star model, reconciled measures, findings as titles
Root cause	15	All causes found, sized, wrong theories rejected
Verdict	10	Answer first, both headlines, specific actions, honest limits

---

12. Common mistakes

- Counting Withdrawn or Pending applications in the approval rate
- Including loans that aren't seasoned in early default
- Joining repayments to applications without adding them up first
- Using `INNER JOIN` to repayments and losing the newest loans
- Assuming one cause explains both headlines
- Comparing counts of defaults instead of default **rates**
- Trusting a rate from a tiny group
- Forgetting that DTI and rates are fractions
- Stopping at the first interesting pattern
- Reporting without a size or an action

Three analyst habits worth building

1. **Big isn't the same as bad.** Whoever has the most defaults may just make the most loans. Compare rates.
2. **When an overall number moves, ask why.** Did every group change? Did one group change? Or did the **mix** of groups shift, so the total moved even though each group stayed the same? Break it down before you explain it.
3. **Compare like with like.** Two groups with different kinds of borrowers will have different results even under the same rules.

---

13. SQL Server 2016 cheat sheet

Need	Use
Trim spaces	`LTRIM(RTRIM(x))`. `TRIM` doesn't exist in 2016.
Text to number safely	`TRY_CONVERT(DECIMAL(12,2), x)`
Remove characters	`REPLACE(x, 'USD', '')`
Text to date (MM/DD/YYYY)	`TRY_CONVERT(DATE, x, 101)`
Text to date (M/D/YY)	`TRY_CONVERT(DATE, x, 1)`
Text to date (YYYY-MM-DD)	`TRY_CONVERT(DATE, x, 23)`
Text to date (Mar 1 2025)	`TRY_CONVERT(DATE, x, 100)`
Add months	`DATEADD(MONTH, 6, d)`
Quarter	`DATEPART(QUARTER, d)`
Bucket a number	`CASE WHEN requested_amount < 10000 THEN 'Under $10K' WHEN requested_amount < 25000 THEN '$10K–$25K' ... END`
One row per loan from many	`GROUP BY loan_id` with `MAX(...)`, `SUM(...)` or `COUNT(...)`
Rank within a group	`RANK() OVER (PARTITION BY a ORDER BY b DESC)`
Avoid integer division	`1.0 * a / b`
Rate from a 1/0 flag	`AVG(1.0 * flag)`
Random sample	`SELECT TOP 50 * FROM t ORDER BY NEWID()`
Remove duplicates	`ROW_NUMBER() OVER (PARTITION BY key ORDER BY ...)` and keep `= 1`
Keep groups above a size	`HAVING COUNT(*) >= 15`

14. When you get stuck

1. **Re-read the definitions.** Especially seasoning, and which decisions count toward approval rate.
2. **Count rows** before and after every join. Repayments are the usual cause of row explosions.
3. **Follow one loan.** Pick one `loan_id` and list its installments in order. Does your early-default flag give the right answer for it?
4. **Make it smaller.** Test on one branch or one product first.
5. **Check the data dictionary.** It says what can be NULL and why.
6. **Ask a classmate about approach.** Each person still submits their own work.
7. **Ask your instructor.** Bring the query, what you expected, and what you got.